PLAYWRIGHT NOTES

1. What is Playwright?

- Playwright is an **open-source automation testing framework**.
- It is developed by **Microsoft**.
- It is mainly used for **automating and testing web applications**.
- It supports major browser engines:
    - **Chromium - chrome , edge**
    - **Firefox**
    - **WebKit - safari**
- It supports multiple programming languages such as:
    - JavaScript / TypeScript
    - Java
    - Python
    - .NET
- Playwright can be used for **end-to-end testing** of web applications.

Interview Definition

> Playwright is an open-source end-to-end test automation framework developed by Microsoft, used to automate and test web applications across Chromium, Firefox, and WebKit browsers.

---

2. Why Do We Use Playwright?

We use Playwright to:

- Automate web application testing.
- Reduce **manual testing effort**.
- Test applications across different browsers.
- Execute tests faster.
- Test complete **end-to-end user workflows**.
- Handle modern and dynamic web applications.
- Run multiple tests in parallel.
- Identify defects early.
- Improve test coverage.
- Reduce repetitive testing work.

> **Note : Flaky scripts : scripts might pass sometime and it might fail sometime(selenium cannot deal with dynamic elements(DOM Structure changes everytime))**

Interview Answer

> We use Playwright to automate web application testing, reduce manual effort, perform cross-browser testing, improve test coverage, and execute repetitive and end-to-end test scenarios efficiently.

---

3. Advantages of Playwright

1. Cross-Browser Support

- Supports **Chromium, Firefox, and WebKit**.
- The same test can be executed across different browsers.

2. Auto-Waiting → most import (30 sec)

- Playwright automatically waits for elements to become ready before performing actions.
- Reduces synchronization issues(speed of execution and speed of the web app) and flaky tests.

3. Fast Execution

- Playwright is designed for fast and efficient test execution.

4. Parallel Execution - imp

- Multiple tests can run simultaneously.
- Reduces overall execution time.

5. Easy Browser Management

- Supports browser, browser context, and page management.
- Allows us to create isolated browser sessions.

6. Multiple Tab/Window Handling - imp

- Can easily handle multiple tabs, windows, and popups.

7. Powerful Debugging

- Provides tools such as:
    - Playwright Inspector
    - Trace Viewer
    - Screenshots
    - Videos
    - HTML reports

8. Supports Modern Web Applications

- Works well with dynamic and modern web applications.

9. Open Source

- Playwright is freely available and has an active developer community.

---

4. Features of Playwright

Important features students should know:

- Cross-browser testing
- Auto-waiting
- Parallel test execution
- Multiple tab and window handling
- Iframe/Frame handling
- Popup handling
- Dialog handling
- Mouse and keyboard actions
- File upload and download
- Screenshots and video recordings
- Assertions
- Hooks
- Annotations
- Test retries
- Test timeout management - 30 sec
- Trace Viewer
- HTML reporting

---

5. Playwright Test Runner → execution takes place `@playwright/test`

Playwright provides its own **built-in test runner**, called **Playwright Test**.

It provides features such as:

- Test execution
- Assertions
- Fixtures
- Hooks
- Parallel execution
- Annotations
- Reporting

Interview Point

> Playwright Test is the built-in test runner provided by Playwright for creating, executing, managing, and reporting automated tests.

---

6. Playwright – Quick Interview Revision

What is Playwright?

> An open-source end-to-end test automation framework developed by Microsoft for testing web applications, api testing, mobile automation.

Why do we use Playwright?

> To automate web application testing, reduce manual effort, perform cross-browser testing, and execute tests efficiently.

Which browsers does Playwright support?

> Chromium(chrome, edge) , Firefox, and WebKit(safari).

Which languages does Playwright support?

> JavaScript, TypeScript, Python, Java, and .NET.

Is Playwright open source?

> Yes, Playwright is an open-source framework developed by Microsoft.

What is Playwright's biggest advantage?

> It provides reliable cross-browser automation with features such as auto-waiting, parallel execution, browser contexts, powerful debugging.

Installation:

1. **Install Node.js** – Playwright requires Node.js and npm.

   Why required? Playwright's JavaScript/TypeScript tooling runs on **Node.js**.
   Use: Provides the **runtime environment** to execute Playwright tests.

   What is it? npm = **Node Package Manager**.
   Why required? Used to **install and manage Playwright and other project dependencies**.
   Example: `npm init playwright@latest`

2. **Install Visual Studio** – For writing test scripts
3. **Check installation** – Verify using `node -v` and `npm -v`.
4. **Create project folder** – Create and open the project in VS Code.
5. **Open Terminal** – Open the terminal inside the project folder.
6. **Initialize Playwright** – Run `npm init playwright@latest`.
   (if any error: npm.cmd init playwright@latest)
7. **Select language** – Choose **JavaScript** or TypeScript.

Playwright folder structure:

```
Playwright Project
│
├── tests/
│   └── example.spec.js
│
├── playwright.config.js
├── package.json
├── package-lock.json
├── node_modules/
```

1. `tests/`

- Main folder where we **store our test cases**.
- Usually contains files ending with `.spec.js` or `.test.js`.

Example: `login.spec.js` / `login.spec.ts`

2. `playwright.config.js`

- The **main configuration file** of Playwright.
- Used to configure things like:
    - Browsers
    - Base URL
    - Timeout
    - Retries
    - Workers
    - Reports
    - Screenshots/videos
    - Projects

---

3. `package.json`

- Contains the **project information and dependencies**.
- Stores Playwright and other npm package details.
- Also contains scripts used to run the project.

---

4. `package-lock.json`

- Keeps the **exact versions of installed npm packages and their dependencies**.
- Helps maintain consistent installations across environments.

---

5. `node_modules/`

- Contains the **installed packages/dependencies** required by the project.
- Created automatically when npm installs packages.

Important: We generally don't manually modify this folder.

Interview Point:

> The Playwright project structure mainly consists of the tests folder for test cases, playwright.config.js for configuration, package.json for dependencies and project scripts, node_modules for installed packages, and report/result folders for test execution artifacts.

First code :

```
import {test} from "@playwright/test"

test("first code" , ()=>{

    console.log("hello");

})
```

Fixtures:

Fixtures are configurations or environments provided by Playwright that provide the required resources for executing a test.

Types of fixtures:

1. **Browser fixture :** Browser fixture is a pre-configured environment provided by Playwright that gives us a browser instance to run our tests.

2. **Context fixture :** Context fixture is a pre-configured environment provided by Playwright that gives us a browser context to run tests in an isolated session, where incognito mode is the default.

3. **Page fixture :** Page fixture is a pre-configured environment provided by Playwright that gives us a tab (page) to interact with and perform actions on a web application.

4. **browserName fixture :** Browser name fixture provides the name of the browser in which the test is being executed, such as Chromium, Firefox, or WebKit.

5. **Request :** This fixture is used in API automation

```
import {test} from "@playwright/test"



//browser fixture

test("fixtures", async({browser})=>{  //tells the browser

    let context = await browser.newContext()  //browser session(isolated
session(profile), incognito mode)

    let page = await context.newPage()  //tab within the broswer session

    await page.goto("https://www.amazon.in/")

}) //control to multiple conext(browser session) mutiple windows
```

```
//context

test ("context fixture", async({context})=>{   //default it will create browser
session in the all the 3 browser

    let page = await context.newPage()

    await page.goto("https://www.amazon.in/")

}) //control over multiple tabs within one session



//page

test("page fixture", async({page})=>{    //tab inside the default 3 browsers

    await page.goto("https://www.amazon.in/")

}) //control over a single tab



//browsername

test("browsername fixture", async({browserName})=>{

    console.log(browserName);

})//return the name of the browser which is in used
```

Execute the test file in headed mode:

npx playwright test tests/example.spec.js --headed

Execute the test file in headless mode:

npx playwright test tests/example.spec.js

Annotation

Annotation is a special instruction used to provide additional information or control the behavior of a test case during execution.

Playwright Annotations

1. `test.skip()`

Definition: Used to **skip the execution of a test case**.

Syntax:

```
test.skip();
```

2. `test.only()`

Definition: Used to **execute only the selected test case** and skip all other tests.

Syntax:

```
test.only("test name", async ({ page }) => {
   // test steps
});
```

3. `test.fail()`

Definition: Used to **mark a test as expected to fail**.

Syntax:

```
test.fail();
```

4. `test.fixme()`

Definition: Used to **mark a test that needs to be fixed and skip its execution**.

Syntax:

```
test.fixme();
```

5. `test.slow()`

Definition: Used to **increase the timeout of a test that is expected to take more time**.

Syntax:

```
test.slow();
```

6. `test.describe()`

Definition: Used to **group related test cases together**.

Syntax:

```
test.describe("group name", () => {

   // test cases

});
```

Commands to run tests:

1. Run all the tests:-
   npx playwright test

2. Run a specific test file:-
   npx playwright test tests/example.spec.js

3. Run all the tests in headed mode:-
   npx playwright test -- headed

4. Run a specific test file in headed mode:-
   npx playwright test tests/example.spec.js -- headed

5. Run only the last failed test:-
   npx playwright test -- last -- failed
   npx playwright test -- last -- failed -- headed

6. Run a test with specific title:-
   npx playwright test -g “your expected title”
   npx playwright test tests/example.spec.js -g “your expected title”

7. Run all the test only in one browser:-
   npx playwright test -- project= chronium
   npx playwright test -- project= firefox
   npx playwright test -- project= webkit

8. Run a specific test in one browser:-
   npx playwright test tests/example.spec.js -- project= chronium
   npx playwright test tests/example.spec.js -- project= firefox
   npx playwright test tests/example.spec.js -- project= webkit

9. Run all the test in multiple browsers:-
   npx playwright test -- project= chronium -- project= webkit

10. Run a specific test in multiple browser:-
    npx playwright test tests/example.spec.js -- project= chronium -- project= webkit

11. Debug test in inspector mode:-

    npx playwright test --debug - -headed

12. To generate report:-

    npx playwright show-report

13. To generate report on specific port no:-

    npx playwright show-report - -port = 9999

Browser Controls & Page Methods

These are some methods which are used to control the browser during the automation.
Example: minimize, maximize, navigation etc

1. `page.goto()` is used to navigate the current page to a specified URL.

   Syntax :

   await page.goto("URL");

2. `page.setViewportSize()` is used to **set or change the width and height of the page's viewport**.

   Syntax:

   await page.setViewportSize({

     width: number,

     height: number })

   ---

3. `page.viewportSize()` is used to **get the current width and height of the page's viewport**.

   Syntax:

   let size = page.viewportSize();

   console.log(size);

---

4. `page.title()` is used to **get the title of the current web page**.

   Syntax:

   let title = await page.title();

---

5. `page.url()` is used to **get the current URL of the page**.

   Syntax:

   let url = await page.url();

---

6. `context.cookies()` is used to **retrieve the cookies stored in the current browser context**.

   Syntax:

   let cookies = await context.cookies();

---

7. `chromium.launch()` is used to **launch a new Chromium browser instance.**

   Syntax:

   let browser = await chromium.launch();

---

8. `browser.newContext()` is used to **create a new isolated browser context (browser session) inside the browser instance**.

   Syntax:

   let context = await browser.newContext();

---

9. `context.newPage()` is used to **create a new page or tab inside the browser context**.

   Syntax:

   let page = await context.newPage();

---

10. `page.screenshot()` is used to **capture a screenshot of the current page**.

    Syntax:

    await page.screenshot({

      path: "foldername/screenshotname.png/jep"

    });

---

11. `browser.close()` is used to **close the browser instance along with all its contexts and pages**.

    Syntax:

    await browser.close();

Examples on browser controls:

```
import {chromium, firefox, test, webkit} from
"@playwright/test"

//page.goto() navigate to a url

test("page.goto()", async({page})=>{

    await page.goto("https://www.amazon.in/")

})

//page.setviewportsize() -- resize the browser

test("page.setviewportsize", async ({page}) => {
```

```
    await page.setViewportSize({width:600 , height:
50000})

    // await page.waitForTimeout(4000)

})

//page.viewportsize()  -- return the size of the browser

test("size of browser", async ({page}) => {

    let size = await page.viewportSize()

    console.log(size);

})

//page.title() -- get the title of the current page

test("get title", async ({page}) => {

    await page.goto("https://www.amazon.in/")

    await page.waitForTimeout(4000)

    let title = await page.title()

    console.log(title);

    // console.log(page.title);

})

//page.url() -- get the url of the current

test("get the url", async ({page}) => {

    await page.goto("https://www.amazon.in/")

    await page.waitForTimeout(4000)

    let url = await page.url()

    console.log(url);

})

//page.screenshot()
```

```
test("screenshot", async ({page}) => {

    await page.goto("https://www.amazon.in/")

    await page.waitForTimeout(3000)

    await page.screenshot({path: "screenshot/ss1.png"})

})

//context.cookies() -- retrieve all cookies from the
current context(browser session) in an array

test("get context cookies" , async ({context}) => {

    let page = await context.newPage()

    await page.goto("https://www.amazon.in/")

    await page.waitForTimeout(3000)

    let cookies = await context.cookies()

    console.log(cookies);

})

//chromium.launch()--launches the chromium browser
instance

test("browser launch", async () => {

    let browser = await chromium.launch()

    // let browser = await webkit.launch()

    // let browser = await firefox.launch()

    let context = await browser.newContext()

    let page = await context.newPage()

    await page.goto("https://www.amazon.in/")

})
```

```
//browser.newcontext() -- used to create a browser
session

test("new context", async({browser})=>{

    let context = await browser.newContext()

//context.page() -- used to create a tab/page withinh the
context

    let page = await context.newPage()

    await page.goto("https://www.amazon.in/")

})

//browser.close()--close the browser instance

test("close browser", async({browser})=>{

    let context = await browser.newContext()

    let page = await context.newPage()

    await page.goto("https://www.amazon.in/")

    await browser.close()

})
```

Locators in Playwright

Locators are used to **identify and locate web elements on a web page** so that Playwright can perform actions on them.

Examples of web elements:

- Text box
- Button
- Link
- Checkbox
- Radio button
- Dropdown

Types of Locators

The commonly used locator approaches are:

1. Locator methods → page.locator(#locator)
    a. CSS selectors
    b. Xpath
2. `getBy` methods

CSS Selector

A **CSS selector** is used to identify a web element based on its HTML attributes, ID, class, tag name, or a combination of these.

Selector Type	Syntax	Example
Class	`.classValue`	`.loginbtn`
ID	`#idValue`	`#submitbtn`
Tag + Class	`tag.classValue`	`button.loginbtn`
Tag + ID	`tag#idValue`	`input#username`
Attribute	`[attribute="value"]`	`[name="email"]`
Tag + Attribute	`tag[attribute="value"]`	`input[name="email"]`

XPath

XPath (XML Path Language) is used to identify and locate web elements in a web page by navigating through the HTML structure and using attributes, text, or other properties of the elements.

Types of XPath

XPath Type	Definition	Syntax	Example
Attribute XPath	Used to locate an element based on its **attribute and attribute value**.	`//tagName[@attribute="value"]`	`//input[@id="username"]`
Text XPath using `text()`	Used to locate an element based on its **visible text** using the `text()` function.	`//tagName[text()="text"]`	`//button[text()="Login"]`
Text XPath using `.`	Used to locate an element based on its **text content** using the `.` operator.	`//tagName[.="text"]`	`//button[.="Login"]`

GetBy Methods in Playwright

GetBy methods are Playwright's user-facing locators used to identify web elements based on how users interact with or see those elements.

They are generally more readable and maintainable than complex CSS or XPath selectors.

---

1. **getByRole()**

`getByRole()` is used to locate an element based on its ARIA role, such as button, textbox, link, checkbox, heading, etc.

Syntax

page.getByRole("role");

Example

await page.getByRole("button", { name: "Login" }).click();

Here, `"button"` is the role and `"Login"` is the accessible name.

---

2. **getByText()**

`getByText()` is used to locate an element based on the visible text displayed on the page.

Syntax

page.getByText("text");

Example

await page.getByText("Welcome to Amazon").click();

---

3. **getByLabel()**

`getByLabel()` is used to locate form elements using their associated label text.

It is commonly used with input fields such as username, email, password, etc.

Syntax

page.getByLabel("label text");

Example

await page.getByLabel("Username").fill("student");

---

4. **getByPlaceholder()**

`getByPlaceholder()` is used to locate an input element using its placeholder text.

Syntax

page.getByPlaceholder("placeholder text");

Example

await page.getByPlaceholder("Enter your email").fill("student@gmail.com");

---

5. **getByAltText()**

`getByAltText()` is used to locate an image or other element using its `alt` attribute value.

Syntax

page.getByAltText("alt text");

Example

await page.getByAltText("Amazon logo").click();

HTML:

<img src="logo.png" alt="Amazon logo">

---

6. **getByTitle()**

`getByTitle()` is used to locate an element using the value of its `title` attribute.

Syntax

page.getByTitle("title text");

Example

await page.getByTitle("Close").click();

HTML:

<button title="Close">X</button>

---

7. **getByTestId()**

`getByTestId()` is used to locate an element using a specific test ID assigned to the element.

By default, Playwright looks for the `data-testid` attribute.

Syntax

page.getByTestId("test-id");

Example

await page.getByTestId("login-button").click();

HTML:

<button data-testid="login-button">Login</button>

Example:

```
import {test} from "@playwright/test"

test("login" , async ({page}) => {

    await
page.goto("https://practicetestautomation.com/practice-test-l
ogin/")

    await
page.locator('input[id="username"]').fill("student")

    await
page.locator('input[id="password"]').fill("Password123")

    await page.locator('button[id="submit"]').click()

})
```

Interview Question:

What are locators in Playwright, and what is the difference between `page.locator()` and `getBy` methods? Give examples of both.

Answer:

Locators are used to identify and interact with web elements on a web page.

`page.locator()` is commonly used with CSS selectors or XPath to locate an element.

await page.locator("#username").fill("student");

`getBy` methods are user-facing locators that identify elements based on how users interact with or see them, such as role, text, label, or placeholder.

await page.getByRole("button", { name: "Login" }).click();

Element Controls in Playwright

Element controls are Playwright methods used to **interact with, retrieve information from, and check the state of web elements**.

1. `fill()`
`fill()` is used to enter text into an input element. It **clears the existing value first** and then enters the new text.

Syntax

await page.locator(".class").fill("text");

Example

await page.locator("#username").fill("admin");

If `"user"` was already present, `fill("admin")` replaces it with `"admin"`.

---

2. `type()`
`type()` is used to type text into an element **character by character**. It can be useful when you want to simulate typing with a delay between characters.

Syntax

await page.locator(".class").type("text", { delay: 500 });

Example

```
await page.locator("#username").type("admin", { delay: 500 });
```

Here, `500` means Playwright waits **500 milliseconds between each character**.

`fill()` vs `type()`

fill() → enters the complete text and replaces existing text

type() → types character by character

---

3. `click()`

`click()` is used to perform a click action on a web element.

It can be used with:

- Buttons
- Links
- Checkboxes
- Radio buttons
- Dropdown options
- Other clickable elements

Syntax

```
await page.locator(".class").click();
```

Example

```
await page.locator("#submit").click();
```

---

4. `innerText()`

`innerText()` returns the **visible text** of an element as it would appear to the user.

It is generally used with a **single element**.

Syntax

```
const text = await page.locator(".class").innerText();
```

Example

HTML:

```
<div>
  Hello
```

World

const text = await page.locator("div").innerText();

console.log(text);

Output:

Hello

The hidden `"World"` is not included.

5. `textContent()`

`textContent()` returns the **text content inside an element**, including text from hidden child elements.

Syntax

const text = await page.locator(".class").textContent();

Using the same HTML:

Hello

World

const text = await page.locator("div").textContent();

console.log(text);

The result contains both:

Hello

World

Easy Difference

innerText() → visible text

textContent() → text content, including hidden text

---

6. `allTextContents()`

`allTextContents()` retrieves the text content of **all matching elements** and returns the results as an **array of strings**.

Syntax

const texts = await page.locator(".class").allTextContents();

Example

Suppose the page contains:

Tom

Jerry

Sam

Code:

const names = await page.locator(".name").allTextContents();

console.log(names);

Output:

["Tom", "Jerry", "Sam"]

---

7. `getAttribute()`

`getAttribute()` retrieves the **value of a specific HTML attribute** from an element.

Syntax

const value = await page.locator(".class").getAttribute("attribute_name");

Example

HTML:

<input id="FirstName" name="firstname">

Code:

const value = await page.locator("#FirstName").getAttribute("name");

console.log(value);

Output:

firstname

---

8. `all()`

`all()` returns **all matching elements as an array of Locator objects**.

Syntax

```
const elements = await page.locator(".class").all();
```

Example

```
const buttons = await page.locator("button").all();

console.log(buttons);
```

9. `selectOption()`

`selectOption()` is used to **select an option from a `<select>` dropdown**.

Syntax

```
await page.locator("select").selectOption("option_value");
```

---

10. `isVisible()`

`isVisible()` checks whether an element is **visible on the page**.

It returns a **boolean value**:

true → element is visible

false → element is not visible

Syntax

```
const result = await page.locator(".class").isVisible();
```

Example

```
const visible = await page.locator("#login").isVisible();
```

console.log(visible);

Output:

true

---

11. `isEnabled()`

`isEnabled()` checks whether an element is **enabled and can be interacted with**.

It returns:

true → element is enabled

false → element is disabled

Syntax

const result = await page.locator(".class").isEnabled();

Example

const enabled = await page.locator("#submit").isEnabled();

console.log(enabled);

---

12. `isChecked()`

`isChecked()` checks whether a **checkbox or radio button** is currently selected/checked.

It returns:

true → checked

false → unchecked

Syntax

const result = await page.locator(".class").isChecked();

Example

const checked = await page.locator("#terms").isChecked();

console.log(checked);

Mouse Actions in Playwright

Mouse actions are used to **simulate mouse interactions** with web elements or specific coordinates on a web page.

1. Click:
`click()` performs a click action on a web element.

By default, Playwright performs a **left-click**.

Syntax

```
await page.locator("locator").click();
```

Or explicitly:

```
await page.locator("locator").click({ button: "left" });
```

Right Click

Used to perform a **right-click** on an element.

```
await page.locator("locator").click({ button: "right" });
```

Middle Click

Used to perform a **middle-click** on an element.

```
await page.locator("locator").click({ button: "middle" });
```

---

2. Double Click:
`dblclick()` performs a **double-click** action on a web element.

Syntax

```
await page.locator("locator").dblclick();
```

You can also perform two clicks using `clickCount`:

await page.locator("locator").click({ clickCount: 2 });

Note: The correct method is `dblclick()`

---

3. Mouse.Move:
`mouse.move()` moves the mouse pointer to a **specific coordinate** on the web page.

Syntax

await page.mouse.move(x, y);

Example

await page.mouse.move(500, 300);

Here:

- `500` → X-coordinate
- `300` → Y-coordinate

Note: The correct syntax is `page.mouse.move()`, not `page.mpuse.move()`.

---

4. Mouse.Down:
`mouse.down()` **presses and holds the mouse button** without releasing it.

Syntax

await page.mouse.down();

It is commonly used with `mouse.move()` for actions such as **dragging**.

Example:

await page.mouse.move(200, 200);

await page.mouse.down();

await page.mouse.move(500, 300);

await page.mouse.up();

---

5. Mouse.Up:
`mouse.up()` **releases the mouse button** after it has been pressed using `mouse.down()`.

Syntax

await page.mouse.up();

Usually used together with:

await page.mouse.down();

await page.mouse.up();

---

6. Hover:
`hover()` moves the mouse pointer over a web element **without clicking it**.

It is commonly used for:

- Displaying dropdown menus
- Showing tooltips
- Revealing hidden elements
- Mouse-over menus

Syntax

await page.locator("locator").hover();

Example

await page.getByText("Products").hover();

---

7. Dispatch Event:
`dispatchEvent()` triggers a **browser event directly on an element** without physically performing the corresponding mouse action.

Syntax

await page.locator("locator").dispatchEvent("click");

Example

await page.locator("#submit").dispatchEvent("click");

Here, Playwright directly triggers the `click` event on the element.

```
import {test} from "@playwright/test"
//right click
test ("right click", async ({page}) => {
    await
page.goto("https://demoapps.qspiders.com/ui/button/buttonRight?sublist=
1")
    await page.locator('button[id="btn_a"]').click({button: "right"})
    await page.locator('//div[text()="No"]').click()
    await page.waitForTimeout(3000)
})
//double click
test("double clcik", async ({page}) => {
    await
page.goto("https://demoapps.qspiders.com/ui/button/buttonDouble?sublist
=2")
    await page.locator('button[id="btn_a"]').dblclick()  //yes
    await page.waitForTimeout(3000)
    await page.locator('button[id="btn_b"]').click({clickCount: 2})
//no
    await page.waitForTimeout(3000)
})
//hover
test("hover action", async ({page}) => {
    await
page.goto('https://demoapps.qspiders.com/ui/mouseHover?sublist=0')
    await
page.locator('img[src="/assets/message-hint-nbRmWGWf.png"]').hover()
    await page.waitForTimeout(3000)
})

//move, down,up
test("move", async ({page}) => {
    await
page.goto("https://demoapps.qspiders.com/ui/dragDrop?sublist=0")
    await page.locator('div[class="cursor-move bg-orange-600 w-36 h-11
p-3 text-white absolute react-draggable"]').hover()
    await page.mouse.down()
    await page.waitForTimeout(3000)
    await page.mouse.move(490,600)
    await page.waitForTimeout(3000)
    await page.mouse.up()
    await page.waitForTimeout(3000)
})
```

```
//dispatch event- deals with disabled elements
test.only("disptach event", async ({page}) => {
    await
page.goto("https://demoapps.qspiders.com/ui/button/buttonDisabled?subli
st=4")
    // await page.locator('input[id="submit"]').dispatchEvent("click")
    await page.locator('input[id="submit"]').click({force:true})
    await page.waitForTimeout(3000)
})
```

Waits in Playwright

Wait is a mechanism that pauses the execution of the test script until a specific condition is met or a defined timeout period passes.

Why Are Waits Necessary?

1. The test script might try to click on a button before it becomes clickable.
2. The test script might attempt to read text from an element that hasn't loaded yet.
3. Test script might fail due to transitions, animations, or slow network response.

Waits help to:

- Reduce flakiness in the test.
- Ensure consistency across different environments.
- Improve reliability.
- Reflect actual user behaviour.

Types of Waits in Playwright

Playwright provides the following types of waits:

1. Auto Waiting

- Smart built-in waits

2. Hard-Coded Waits

- Static waits

3. Explicit Waits

3.1 Element-Based Waits

- Wait for text
- Wait for timeout
- Wait for selector
- Wait for element state

3.2 Page-Level Waits

- Navigation wait
- Load state wait

3.3 Custom Waits

1. Auto Waiting

- Auto waiting is a **built-in Playwright feature**.
- It waits automatically for an element to be ready before performing an action.
- Common actions that support auto-waiting:

click()
fill()
type()
check()
hover()
dblclick()
uncheck()

- Playwright automatically waits for up to **30 seconds** by default.

1. Page-Level Timeout

await page.setDefaultTimeout(40000);

- Sets the **default action timeout** to 40 seconds for that page.
- Applies to actions such as `click()`, `fill()`, `check()`, `hover()`, etc.

---

2. Individual Action Timeout

await page.locator("#login").click({ timeout: 40000 });

- Sets the timeout to **40 seconds for that particular action only**.
- Other actions will continue using their existing timeout.

---

3. Test-Level Action Timeout

test.use({ actionTimeout: 50000 });

- Sets the **default action timeout to 50 seconds for the test**.
- Applies to Playwright actions performed within that test.

```
//all the action in the script

test.use({actionTimeOut:50000})

test("right click", async ({page}) => {

    //change the default timeout

    await page.setDefaultTimeout(40000)

    await
page.goto("https://demoapps.qspiders.com/ui/button/buttonRight?sublist=
1")

    await page.locator('button[id="btn_a"]').click({button: "right"})

    //for individual action

    await page.locator('//div[text()="No"]').click({timeout:40000})
//30sec

    await page.waitForTimeout(3000)

})
```

---

2. Hard-Coded Waits / Static Waits

- A hard-coded wait is a **fixed delay** where Playwright pauses the execution for a specific amount of time.
- It does not wait for a specific condition.

Syntax
await page.waitForTimeout(4000);

Here, Playwright pauses the execution for **4000 milliseconds (4 seconds)**.

Disadvantages

- Increases the execution time.
- Makes the test slower.
- Not recommended for normal test execution.
- Cannot cope with the actual speed of the application.

When to Use

Hard-coded waits can be used for:

- Debugging the test.
- Troubleshooting timing issues.

```
// await
page.locator('input[id="submit"]').dispatchEvent("click")

    await page.locator('input[id="submit"]').click({force:true})

    await page.waitForTimeout(3000)
```

3. Explicit Waits in Playwright

Explicit waits are useful when the application performs an action dynamically and we need to wait for a particular condition.

Explicit Waits Can Be Used For

- Waiting for an element to be **present, visible, hidden, or detached**.
- Waiting for a specific **text or data** to appear.
- Waiting for **page navigation** to complete.
- Waiting for a particular **page load state**.
- Waiting for a specific **event** to occur.
- Waiting for a custom condition to become true.

Types of Element-Based Waits:

1. Wait for Text

Used to wait until a specific text appears inside an element.

2. Wait for Timeout

Used to pause the test execution for a specific amount of time.

3. Wait for Selector

Used to wait for an element matching a CSS/XPath selector to reach a specific state.

4. Wait for Element State

Used to wait for a locator to reach a specific state.

Common states:

visible → Waits until the element is **present in the DOM and visible** to the user.
attached → Waits until the element is **present/attached to the DOM**.
hidden → Waits until the element is **hidden or not visible**.
detached → Waits until the element is **removed from the DOM**.

```
import {test} from "@playwright/test"
//wait for text
test("wait for text", async ({page}) => {
    await page.goto("https://shoppersstack.com/")
    await page.locator('//h3[text()="Welcome to ShoppersStack. Enjoy shopping with
us."]',
        {hasText:"Welcome to ShoppersStack. Enjoy shopping with us."}).waitFor()

    await page.locator('//button[text()="Login"]', {hasText:'Login'}).waitFor()
})

//wait for timeout -- give waits for a specific element
test("wait for timeout", async ({page}) => {
    await page.goto("https://shoppersstack.com/")
    await page.locator('//button[text()="Login"]').waitFor({timeout:50000,
state:'attached'})
})

//wait for selector
test("wait for selector", async ({page}) => {
     await page.goto("https://shoppersstack.com/")
     await page.waitForSelector('//h3[text()="Welcome to ShoppersStack. Enjoy
shopping with us."]')
     await page.waitForSelector('//button[text()="Login"]')
})

//wait for state
test("wait for state", async ({page}) => {
    await page.goto("https://shoppersstack.com/")
    await page.locator('//button[text()="Login"]').waitFor({state:'attached'}) //30
sec
})
```

Types of Page Level Waits:

Wait for Navigation

`waitForNavigation()` is used to wait until a page navigation is completed before continuing the test execution.

Syntax
await page.waitForNavigation();

```
//wait for navigation
test.only('wait for navigation', async ({browser}) => {
    let context = await browser.newContext()
    let page = await context.newPage()
    await page.goto('https://demoapps.qspiders.com/ui/browser?sublist=0')
    let [nav] = await Promise.all([
       page.waitForNavigation(),
       page.locator('//button[text()="view more"]').first().click()
    ])
})
```